

4-Hour Bridge Session: NumPy, Pandas & Matplotlib for Data Interpretation

A hands-on Google Colab workshop leading into “A2: Data Interpretation and Visualization using Python” (BA Prog, Sem II)

1. Session Objective

This is a **pre-course bridge session** for students about to start the attached syllabus. The goal is not to teach the full syllabus in one sitting, but to get every student comfortable and confident with the three core libraries — **NumPy, Pandas, Matplotlib** — through two complete, realistic case studies, so that Units 1–4 of the course feel like a continuation rather than a cold start.

By the end of the session, students will be able to:

- Create, index, slice and reshape NumPy arrays
- Load, clean, filter, group and summarize data with Pandas
- Produce labelled, legended plots with Matplotlib
- Explain, at a working level, how these three libraries feed into a simple ML workflow (clustering and classification) — this previews Unit 1’s mention of statistical foundations and sets up later practicals (esp. Practicals 2 and 3, which use the Iris and Titanic datasets)

Dataset used throughout: Iris (built into scikit-learn) as the spine of the session, with a short Titanic detour for Naive Bayes classification — deliberately the same two datasets the official Practical List already asks students to use, so this session is direct preparation for it.

Platform: Google Colab (no local installs; numpy , pandas , matplotlib , scikit-learn , seaborn are preinstalled).

2. Time Allocation Overview

#	Segment	Duration	Cumulative
0	Welcome, Colab orientation, dataset intro	15 min	0:15
1	NumPy — arrays, indexing, reshaping	40 min	0:55
2	Pandas — Series, DataFrame, cleaning, groupby	55 min	1:50
—	Break	10 min	2:00
3	Matplotlib — plotting fundamentals	40 min	2:40
4	Case Study 1: K-Means Clustering on Iris	45 min	3:25
5	Case Study 2: Naive Bayes on Titanic	25 min	3:50
6	Wrap-up, syllabus mapping, Q&A	10 min	4:00

3. Detailed Run-of-Show

Segment 0 — Welcome & Setup (15 min)

- Open a shared Colab notebook (link it ahead of time; give a blank copy too, “File → Save a copy in Drive”).
- Quick tour: cells, `Shift+Enter`, runtime type, `!pip install` if ever needed.
- Load the datasets once up front so they’re visible all session:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris

iris = load_iris(as_frame=True)
df = iris.frame
df['species'] =
df['target'].map(dict(enumerate(iris.target_names)))
df.head()
```

- Frame the story: “We’ll clean, understand, plot, then let the machine group and classify — the same arc as your course.”
-

Segment 1 — NumPy (40 min)

Maps to syllabus: Ch 4 (4.1) — array creation, indexing, slicing, swapping axes, transposing.

1. Creating arrays (10 min)

```
a = np.array([1, 2, 3, 4, 5])
b = np.array([[1, 2, 3], [4, 5, 6]])
zeros = np.zeros((3, 3))
ones = np.ones((2, 4))
rand_arr = np.random.rand(3, 3)      # rand()
randn_arr = np.random.randn(3, 3)    # rand()-
equivalent
rand_int = np.random.randint(0, 100, size=(3, 3)) #
randint()
```

2. Indexing & slicing (10 min) — pull rows/columns from the Iris feature matrix converted to a NumPy array:

```
X = iris.data.values
X[0]      # first row
X[:, 0]   # first column (sepal length)
X[0:5, 0:2] # first 5 rows, first 2 columns
```

3. Swapping axes / transposing (10 min):

```
X.T      # transpose
X.swapaxes(0, 1) # same effect for 2D
```

4. Data processing with NumPy (10 min) — vectorized stats to foreshadow Pandas:

```
X.mean(axis=0), X.std(axis=0), X.min(axis=0),
X.max(axis=0)
```

Micro-exercise: “Create a 4×4 random integer array, print its transpose, and compute the column-wise mean.” (5 min built into above, or as a quick-fire poll)

Segment 2 — Pandas (55 min)

Maps to syllabus: Ch 5 (5.1–5.3), Ch 6 (6.1), Ch 7 (7.1–7.2), Ch 8 (8.1–8.3) — Series/DataFrame, arithmetic, statistics, reindexing, filtering, missing data, wrangling.

1. Series & DataFrame basics (10 min)

```
s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])
df.dtypes
df.describe()
df.info()
```

2. Indexing, reindexing, filtering (10 min)

```
df.loc[df['species'] == 'setosa']
df.iloc[0:5, 0:3]
df[df['sepal length (cm)'] > 5.5]
```

3. Handling missing data (10 min) — deliberately inject some NaNs to demonstrate the workflow used in Practical 1:

```
df_dirty = df.copy()
df_dirty.loc[2:4, 'petal width (cm)'] = np.nan
df_dirty.isnull().sum()
df_dirty.dropna()
df_dirty.fillna(df_dirty.mean(numeric_only=True))
```

4. Duplicates & outliers (10 min) — directly rehearses Practical 1(a)(b):

```
df_dirty.duplicated().sum()
df_dirty.drop_duplicates()

Q1 = df['petal length (cm)'].quantile(0.25)
Q3 = df['petal length (cm)'].quantile(0.75)
IQR = Q3 - Q1
outliers = df[(df['petal length (cm)'] < Q1 - 1.5*IQR) |
(df['petal length (cm)'] > Q3 + 1.5*IQR)]
```

5. GroupBy, aggregation, correlation (15 min) — previews Unit 5 (Weeks 14–15) and Practical 1(c)/2(ii):

```
df.groupby('species').mean(numeric_only=True)
df.groupby('species').agg(['mean', 'std'])
df.corr(numeric_only=True)
pd.crosstab(df['species'], pd.cut(df['petal length (cm)'], 3))
```

Micro-exercise: “Group by species and find which numeric column has the largest spread (std).”

Break (10 min)

Segment 3 — Matplotlib (40 min)

Maps to syllabus: Ch 9 (9.1–9.2), Ch 5 (pg 281–282) — figures, subplots, labels/legends, bar, scatter, histogram, heatmap.

1. Basic figure & scatter (10 min)

```
plt.figure(figsize=(6,4))
plt.scatter(df['sepal length (cm)'], df['sepal width (cm)'], c=df['target'], cmap='viridis')
plt.xlabel('Sepal Length (cm)')
```

```
plt.ylabel('Sepal Width (cm)')
plt.title('Sepal Length vs Width by Species')
plt.legend(*plt.gca().collections[-1].legend_elements(),
title='Species')
plt.show()
```

2. Subplots (10 min)

```
fig, axes = plt.subplots(1, 2, figsize=(10,4))
axes[0].hist(df['petal length (cm)'], bins=15,
color='teal')
axes[0].set_title('Petal Length Distribution')
axes[1].boxplot(df['sepal width (cm)'])
axes[1].set_title('Sepal Width Spread')
plt.tight_layout()
plt.show()
```

3. Bar chart & heatmap (15 min) — heatmap directly rehearses Practical 2(ii):

```
df.groupby('species').mean(numeric_only=True).plot(kind='bar')

plt.title('Mean Feature Values by Species')
plt.ylabel('cm')
plt.show()

import seaborn as sns
plt.figure(figsize=(6,5))
sns.heatmap(df.corr(numeric_only=True), annot=True,
cmap='coolwarm')
plt.title('Feature Correlation Heatmap')
plt.show()
```

4. Quick pairplot teaser (5 min) — links straight to Practical 3(d):

```
sns.pairplot(df, hue='species')
```

Segment 4 — Case Study 1: K-Means Clustering on Iris (45 min)

Purpose: show NumPy (math under the hood) + Pandas (data prep) + Matplotlib (visual validation) working together, and preview the “statistical hypothesis” spirit of Unit 1 in an unsupervised setting.

1. **Why clustering, in one minute** (5 min) — “We hide the species label and ask: can the algorithm rediscover 3 natural groups just from measurements?”
2. **Prepare the data** (5 min)

```
from sklearn.preprocessing import StandardScaler
X = df.drop(columns=['target', 'species'])
X_scaled = StandardScaler().fit_transform(X) # NumPy
array under the hood
```

3. Run K-Means (10 min)

```
from sklearn.cluster import KMeans
kmeans = KMeans(n_clusters=3, random_state=42,
n_init=10)
clusters = kmeans.fit_predict(X_scaled)
df['cluster'] = clusters
```

4. Visualize & validate against ground truth (15 min)

```
fig, axes = plt.subplots(1, 2, figsize=(11,4))
```

```

axes[0].scatter(X['petal length (cm)'], X['petal width (cm)'], c=df['cluster'], cmap='viridis')
axes[0].set_title('K-Means Clusters')
axes[1].scatter(X['petal length (cm)'], X['petal width (cm)'], c=df['target'], cmap='viridis')
axes[1].set_title('True Species Labels')
for ax in axes:
    ax.set_xlabel('Petal Length (cm)')
    ax.set_ylabel('Petal Width (cm)')
plt.tight_layout()
plt.show()

pd.crosstab(df['species'], df['cluster']) # confusion-style cross-tab

```

5. **Discussion** (10 min) — how close were the clusters to real species? What role did scaling play? (Ties back to Pandas groupby / crosstab learned earlier.)

Segment 5 — Case Study 2: Naive Bayes on Titanic (25 min)

Purpose: contrast unsupervised vs. supervised learning, and directly rehearse Practical 3 & 4 (Titanic-based exercises).

1. **Load & quick-clean Titanic** (10 min)

```

titanic = sns.load_dataset('titanic')
titanic_clean =
titanic[['survived', 'pclass', 'sex', 'age', 'fare']].dropna()

titanic_clean['sex'] =
titanic_clean['sex'].map({'male':0, 'female':1})

```

2. **Train/test split & fit Naive Bayes** (10 min)

```

from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score,
confusion_matrix

X = titanic_clean[['pclass', 'sex', 'age', 'fare']]
y = titanic_clean['survived']
X_train, X_test, y_train, y_test = train_test_split(X,
y, test_size=0.25, random_state=42)

model = GaussianNB()
model.fit(X_train, y_train)
preds = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, preds))

```

3. **Visualize the result** (5 min) — a bar chart of survival rate by sex/class, directly matching Practical 3(a):

```

titanic_clean.groupby(['pclass', 'sex'])
['survived'].mean().unstack().plot(kind='bar')
plt.ylabel('Survival Rate')
plt.title('Survival Rate by Class and Sex')
plt.show()

```

Segment 6 — Wrap-up & Syllabus Mapping (10 min)

- Recap: “Today you used NumPy for the math, Pandas for the data wrangling, and Matplotlib for the story — that’s exactly the sequence Units 2, 3 and 4 of your course will go into in

much more depth.”

- Show a one-slide mapping table (below) so students see today wasn’t a detour — it’s a preview.
- Point students to the official Practical List questions 1–5 as “homework you can now attempt.”
- Open floor for questions; collect one or two topics students feel shakiest on, for the instructor to revisit in Week 1 of the actual course.

4. Mapping Today’s Session to the Official Syllabus

Today’s Segment	Syllabus Unit	Chapter Reference
NumPy arrays, indexing, transpose	Unit 2 (Weeks 3–5)	Ch 4: 4.1
Pandas Series/DataFrame, cleaning, missing data	Unit 3 (Weeks 6–9)	Ch 5: 5.1–5.3; Ch 7: 7.1–7.2; Ch 8: 8.1–8.3
Matplotlib scatter, bar, subplots, heatmap	Unit 4 (Weeks 10–13)	Ch 9: 9.1–9.2
GroupBy, crosstab (used in Case Study 1)	Unit 5 (Weeks 14–15)	Ch 10: 10.1–10.4
Descriptive stats, correlation (Segment 2 & Case Study 1)	Unit 1 (Weeks 1–2)	Ch 1: pg 11–38

5. Materials Checklist

- ☐ Colab notebook link shared in advance (view-only + “make a copy” instructions)
- ☐ Confirm `seaborn` and `scikit-learn` load without extra installs on the day
- ☐ Backup: download `iris.csv` and `titanic.csv` locally in case `sklearn / seaborn` dataset servers are unreachable
- ☐ Printed/shared copy of the official Practical List (1–5) to hand out at the end
- ☐ Timer visible to instructor to hold segment lengths (this plan is tight — a 5–10 min buffer is built into Segment 0 and the break)

6. Facilitation Notes

- Keep code cells short and runnable — type live rather than paste, so students see it build up.
- After each segment, pause for one “what did we just do, in one sentence” recap before moving on — this is what makes the bridge into the full course feel continuous.
- If time runs short, the safest segment to compress is Segment 5 (Naive Bayes) — Segment 4 (K-Means) is the richer demonstration since it uses all three libraries together.